

PRACTICAL 6

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    int request_fd, response_fd;
    char message[100];
    char response[150];

    // Create FIFOs
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

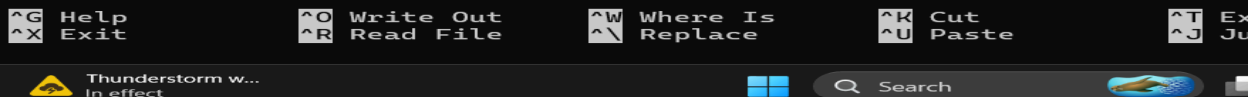
    printf("Server started...\n");
    printf("Waiting for client message...\n");

    // Open request FIFO for reading
    request_fd = open(REQUEST_FIFO, O_RDONLY);

    // Open response FIFO for writing
    response_fd = open(RESPONSE_FIFO, O_WRONLY);

    while (1)
    {
        memset(message, 0, sizeof(message));

        if (read(request_fd, message, sizeof(message)) > 0)
        {
            printf("Client: %s", message);
        }
    }
}
```



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    int request_fd, response_fd;
    char message[100];
    char response[150];

    // Open request FIFO for writing
    request_fd = open(REQUEST_FIFO, O_WRONLY);

    // Open response FIFO for reading
    response_fd = open(RESPONSE_FIFO, O_RDONLY);

    printf("Client connected to server.\n");

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        // Send message to server
        write(request_fd, message, strlen(message));

        // Receive response from server
        memset(response, 0, sizeof(response));
        read(response_fd, response, sizeof(response));

        printf("Server: %s", response);

        if (strncmp(message, "exit", 4) == 0)
            break;
    }
}
```

```
ruhi0404@Ruhitha:~$ ./prac6task1_server
Server started...
Waiting for client message...
Client: Hello Server
Response sent to client.
Waiting for client message...

ruhi0404@Ruhitha:~$ ./prac6task1_client
Client connected to server.
Enter message: Hello Server
Server: Server processed: Hello Server
Enter message:
```

Observation

1. Two Named Pipes (FIFOs) were created for communication between the client and server.
2. The client successfully sent messages to the server through the request FIFO.
3. The server read the messages, processed them, and sent the response back to the client through the response FIFO.
4. Communication was bidirectional: Client → Server and Server → Client.
5. The FIFO follows the First-In-First-Out (FIFO) principle, so messages are generally read in the order in which they are written.
6. When multiple clients communicate with the same server, their requests are received through the common FIFO and are handled according to the order of their writes.
7. FIFO communication provides synchronization between processes because a process may wait until another process opens/writes to the FIFO.
8. After the client sends `exit`, the server sends a termination response and closes the FIFOs.

Task 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nReceived SIGINT (Ctrl+C)\n");
        printf("Signal handler executed for SIGINT.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
        printf("Signal handler executed for SIGTERM.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
        printf("Signal handler executed for SIGUSR1.\n");
    }

    printf("Waiting for another signal...\n");
}

int main()
{
    printf("POSIX Signal Handling Program\n");
    printf("Process ID: %d\n", getpid());

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handlers registered successfully.\n");
    printf("Waiting for signals...\n");

    while (1)
    {
        pause();
    }
}
```

```
[5]+ Stopped ./prac6task2
ruhi0404@Ruhitha:~$ ./prac6task2
POSIX Signal Handling Program
Process ID: 2161
Signal handlers registered successfully.
Waiting for signals...
^C
Received SIGINT (Ctrl+C)
Signal handler executed for SIGINT.
Waiting for another signal...

Received SIGUSR1
Signal handler executed for SIGUSR1.
Waiting for another signal...
|
```

```
ruhi0404@Ruhitha:~$ cd ~
ruhi0404@Ruhitha:~$ kill -SIGUSR1 2161
```

Observation

1. A POSIX signal handling program was successfully implemented using the `signal()` system call.
2. Signal handlers were registered for SIGINT, SIGTERM, and SIGUSR1.
3. SIGINT was generated using Ctrl+C and was successfully caught by the signal handler.
4. SIGUSR1 and SIGTERM were sent to the process using the `kill` command.
5. Whenever a signal was received, the corresponding signal handler was executed immediately.
6. The program continued its execution after handling the signals instead of terminating.
7. This demonstrates asynchronous event handling, because signals can interrupt a process at any time and cause the registered handler to execute.
8. The signal handler is a function that specifies the action to be performed when a particular signal is received.
9. The process ID (PID) is required to send signals to a specific running process using the `kill` command.